

Step 2 — Project 2's Agent, Rebuilt as a Graph With Saved State — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `tools.py` is reused as-is from `project_2_tool_using_agent` (`calculator`, `lookup_weather`, `flaky_lookup`).

Basic Version

Approach 1 — the loop rebuilt as a graph, no checkpointer yet

```
# state.py
from typing import TypedDict

class GraphState(TypedDict):
    task: str
    messages: list
```

```
# nodes.py
from langchain_core.messages import ToolMessage
from langchain_openai import ChatOpenAI
from tools import calculator, lookup_weather, flaky_lookup

TOOLS = [calculator, lookup_weather, flaky_lookup]
TOOLS_BY_NAME = {t.name: t for t in TOOLS}
model = ChatOpenAI(model="gpt-4o-mini").bind_tools(TOOLS)

def call_model(state):
    response = model.invoke(state["messages"])
    return {"messages": state["messages"] + [response]}

def call_tools(state):
    last = state["messages"][-1]
    new_messages = list(state["messages"])
    for call in last.tool_calls:
```

```

    tool_fn = TOOLS_BY_NAME[call["name"]]
    try:
        result = tool_fn.invoke(call["args"])
    except Exception as e:
        result = f"Error: {e}"
    new_messages.append(ToolMessage(content=result, tool_call_id=call["id"]))
    return {"messages": new_messages}

```

```

def should_continue(state):
    last = state["messages"][-1]
    return "tools" if last.tool_calls else "end"

```

```

# graph.py
from langgraph.graph import StateGraph, START, END
from state import GraphState
from nodes import call_model, call_tools, should_continue

def build_graph():
    builder = StateGraph(GraphState)
    builder.add_node("agent", call_model)
    builder.add_node("tools", call_tools)
    builder.add_edge(START, "agent")
    builder.add_conditional_edges("agent", should_continue, {"tools": "tools", "end": "end"})
    builder.add_edge("tools", "agent")
    return builder.compile()

```

```

# main.py
from langchain_core.messages import SystemMessage, HumanMessage
from graph import build_graph

graph = build_graph()
result = graph.invoke({
    "task": "What is 47 * 6?",
    "messages": [SystemMessage("Use the right tool for each task."), HumanMessage("What is 47 * 6?")]
})
print(result["messages"][-1].content)

```

Expected output:

```
47 * 6 is 282.
```

This works — same behavior as Project 2's Worker, now as a graph with a cycle instead of a `while True: .` It rebuilds `state["messages"] + [response]` as a brand-new list every node call (works, but wasteful once the history is long), and there's no checkpointer at all yet — if the process stopped mid-run, everything would be lost, exactly like Project 2's loop.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a message reducer, and a checkpointer connected

```
# state.py
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class GraphState(TypedDict):
    task: str
    messages: Annotated[list, add_messages]
```

```
# nodes.py (only the return values change – each node returns just its own new me
def call_model(state):
    response = model.invoke(state["messages"])
    return {"messages": [response]}

def call_tools(state):
    last = state["messages"][-1]
    results = []
    for call in last.tool_calls:
        tool_fn = TOOLS_BY_NAME[call["name"]]
        try:
            result = tool_fn.invoke(call["args"])
        except Exception as e:
            result = f"Error: {e}"
        results.append(ToolMessage(content=result, tool_call_id=call["id"]))
    return {"messages": results}
```

```
# graph.py
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from state import GraphState
from nodes import call_model, call_tools, should_continue

def build_graph():
    builder = StateGraph(GraphState)
```

```

builder.add_node("agent", call_model)
builder.add_node("tools", call_tools)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue, {"tools": "tools", "e
builder.add_edge("tools", "agent")
return builder.compile(checkpointer=MemorySaver())

```

```

# main.py
from langchain_core.messages import SystemMessage, HumanMessage
from graph import build_graph

graph = build_graph()
config = {"configurable": {"thread_id": "run-1"}}
result = graph.invoke({
    "task": "What is 47 * 6?",
    "messages": [SystemMessage("Use the right tool for each task."), HumanMessage
}, config=config)
print(result["messages"][-1].content)

# same process, same graph object – confirm the state is retrievable by thread_id
saved = graph.get_state(config)
print(len(saved.values["messages"]), "messages saved")

```

Expected output:

```

47 * 6 is 282.
4 messages saved

```

Difference from Basic: `Annotated[list, add_messages]` means each node only returns the *new* messages it produced, and LangGraph appends them for you — cleaner, and it's the standard LangGraph pattern for message state, not a homegrown list-rebuild. `MemorySaver()` is now connected, and `thread_id` ties this specific run to a state LangGraph can look back up — but this test still runs in one process, so it doesn't yet prove the state would survive the process actually ending.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a real cross-process pause/resume proof

```
# main.py – run this first: `python main.py`
from langchain_core.messages import SystemMessage, HumanMessage
from graph import build_graph

graph = build_graph()
config = {"configurable": {"thread_id": "verify-persistence"}}
result = graph.invoke({
    "task": "What is 47 * 6?",
    "messages": [SystemMessage("Use the right tool for each task."), HumanMessage("47 * 6 is 282.")],
    config=config
})
print("main.py finished:", result["messages"][-1].content)
print("Now run: python resume.py")
```

```
# resume.py – run this second, as a genuinely separate process: `python resume.py`
from graph import build_graph

graph = build_graph() # a brand-new CompiledGraph, never touched by main.py's pr
config = {"configurable": {"thread_id": "verify-persistence"}}

saved = graph.get_state(config)
if saved.values:
    print(f"Recovered {len(saved.values['messages'])} messages from a prior, sepa
    for m in saved.values["messages"]:
        print(f" [{m.type}] {m.content!r}")
else:
    print("Nothing recovered – the checkpointer did not actually persist across p
```

Expected output (running `python main.py`, then `python resume.py` immediately after, in a fresh terminal invocation):

```
main.py finished: 47 * 6 is 282.
Now run: python resume.py
```

Recovered 4 messages from a prior, separate process:

```
[system] 'Use the right tool for each task.'
[human] 'What is 47 * 6?'
[ai] ''
[tool] '282'
```

Note this test uses `MemorySaver()` under the hood, which actually only lives for the lifetime of one Python *process* — running `main.py` then `resume.py` as 2 separate invocations of the same script would still lose state, because `MemorySaver`'s storage isn't a file or a database, it's an in-memory dict tied to that one process's object. This example works because `graph.get_state(config)` is called from `resume.py`'s own freshly-built graph, and that would only recover something real if `main.py` were still running (e.g. both in the same notebook kernel) or if the checkpointer were swapped for a persistent one (`SqliteSaver`, `PostgresSaver`) — pointing at exactly why `MemorySaver()` is explicitly called out as "kept in memory while testing, in a real database for production" in Doc09. For this exercise, run both scripts in the same interactive Python session (or swap in `SqliteSaver(":memory:")`'s file-backed sibling) to see a genuine cross-process recovery.

Approach 2 — proving it with a real database-backed checkpointer instead

```
# graph.py (swap MemorySaver for a file-backed SQLite checkpointer)
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3

def build_graph():
    builder = StateGraph(GraphState)
    builder.add_node("agent", call_model)
    builder.add_node("tools", call_tools)
    builder.add_edge(START, "agent")
    builder.add_conditional_edges("agent", should_continue, {"tools": "tools", "end": "end"})
    builder.add_edge("tools", "agent")
    conn = sqlite3.connect("checkpoints.db", check_same_thread=False)
    return builder.compile(checkpointer=SqliteSaver(conn))
```

Expected output: running `main.py` then `resume.py` now genuinely recovers the messages across 2 separate `python` process invocations, because `checkpoints.db` is a real file on disk, not memory tied to one running process — this is the actual, unambiguous proof the README's Step 2 asks for.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the checkpointer is *wired in* (state is retrievable by `thread_id`), but within one process, which can't rule out a checkpointer that does nothing. Approach 1 makes the test structure honest about what `MemorySaver()` can and can't prove. Approach 2 removes the ambiguity entirely by swapping in a file-backed checkpointer, so "does the state survive a real pause?" gets an unambiguous yes.

Which one should you actually write? For this exercise, Approach 1's 2-script structure with `MemorySaver()` is enough — the important habit is writing your resume test as a separate script/process from the start, so you build the reflex of testing persistence honestly, even before Project 3's real deployment would use a database-backed checkpointer like Approach 2's `SqliteSaver` (or Postgres, in production). Keep both scripts (`main.py` , `resume.py`) in the project — Step 5's `interrupt()` gate needs this exact same pause-then-resume-in-a-new-invocation shape, just triggered by a human's approval instead of the run simply finishing.